

PEMDOS

Persistent Erasure-coded Modular Distributed Object Store

CS 588 - Project Report - Siddharth Ganesh

1. Introduction

PEMDOS is a distributed object storage system that splits files across five independent storage nodes using Reed-Solomon erasure coding, and verifies their integrity using the homomorphic fingerprinting scheme described in:

Hendricks, J., Ganger, G. R., & Reiter, M. K. (2007). Verifying Distributed Erasure-Coded Data. Proceedings of the 26th ACM Symposium on Principles of Distributed Computing (PODC).

The system tolerates the simultaneous failure or silent corruption of up to two storage nodes. Any three of the five shards are sufficient to reconstruct the original file. The paper's central contribution: a verification scheme that detects when a shard is inconsistent with the original encoding of the file, implemented from scratch over $GF(2^8)$.

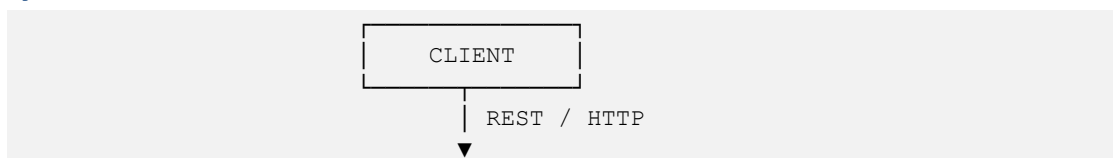
1.1 The Problem

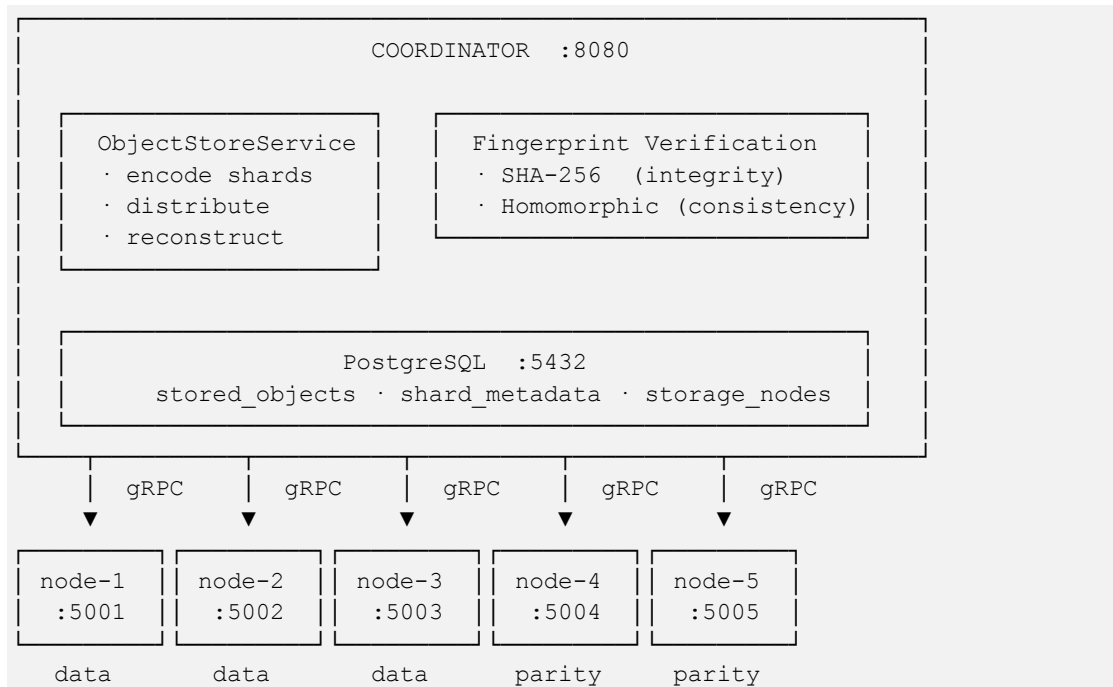
In a distributed storage system with untrusted nodes, three failure modes must be handled:

1. Node failure: a node goes offline; its shards are unreachable.
2. Bit-level corruption: a shard's bytes are altered (disk error, network flip).
3. Silent inconsistency: a node returns a shard whose bytes are internally well-formed but do not match what was originally written.

Replication addresses (1) and (2) but is wasteful (3× overhead). Reed-Solomon erasure coding addresses (1) and (2) at lower overhead (1.67× for 3+2). Neither addresses (3) directly. A hash like SHA-256 only detects whether a shard differs from its recorded hash, not whether the shard is consistent with the rest of the encoding. Hendricks et al. solve this with a homomorphic fingerprint that is provably linked to the encoding matrix.

2. System Architecture





2.1 Components

Component	Role
Coordinator	Receives client requests, encodes/decodes shards, distributes them, verifies integrity, persists metadata
Storage nodes (×5)	gRPC servers that store/retrieve/delete shard files on local disk
PostgreSQL	Stores object metadata, per-shard fingerprints, eval point r, node registry
Common library	Shared codec, Galois field arithmetic, fingerprint logic, protobuf definitions

2.2 Module Layout

The project is structured as three Gradle submodules:

- common/ -> shared library: Reed-Solomon codec, $GF(2^8)$ arithmetic, homomorphic fingerprint, protobuf definitions
- coordinator/ -> Spring Boot REST service, JPA entities, gRPC client, dashboard UI
- storage-node/ -> Spring Boot gRPC server, file-based shard storage

3. Design Choices

3.1 Reed-Solomon parameters: 3 data + 2 parity

We chose 3+2 because it is the minimum configuration that tolerates two simultaneous failures while keeping storage overhead low ($1.67\times$ vs $3\times$ for full replication). The fault tolerance

threshold becomes a hard mathematical boundary: any 3 of 5 shards suffice; any fewer is unrecoverable. This boundary is demonstrable in a live demo.

3.2 GF(2⁸) over GF(2¹⁶)

GF(2⁸) was chosen because each field element fits in exactly one byte. There is no packing or precision loss, and shards are processed byte-by-byte without conversion. The trade-off is a smaller field (256 elements vs 65536), which limits the maximum number of shards to 255, far beyond our 5-node setup, so not a constraint in practice.

3.3 Spring Boot + REST for the coordinator

The coordinator exposes a REST API that any client (curl, browser, library) can use without code generation. Spring Boot was chosen for its mature dependency injection, JPA integration, and minimal boilerplate. The dashboard is served as a single static index.html from the same Spring Boot instance; no separate frontend server is needed.

3.4 gRPC for coordinator-node communication

Internal communication uses gRPC instead of REST for three reasons:

4. Type safety: the .proto schema is the single source of truth; both the coordinator's client stubs and the node's server skeletons are generated from it.
5. Efficient binary transport: shard payloads are raw bytes; protobuf avoids the base64 overhead of JSON.
6. Per-call deadlines: gRPC has first-class support for deadlines, which we use heavily to prevent dead nodes from blocking the coordinator (every gRPC call is capped at 500ms).

3.5 PostgreSQL for metadata

The metadata schema is purely relational (objects → shards → nodes). PostgreSQL via JPA/Hibernate is a natural fit. Spring Boot's ddl-auto: update keeps the schema in sync with entity definitions during development.

3.6 Owning the Reed-Solomon codec instead of using a library

The homomorphic fingerprint scheme requires the exact coefficients of the encoding matrix to compute the expected fingerprint of each shard:

$$\text{fp}(\text{shard_j}) = M[j][0] \cdot \text{fp}(D0) + M[j][1] \cdot \text{fp}(D1) + M[j][2] \cdot \text{fp}(D2)$$

A third-party library (e.g. Backblaze JavaReedSolomon) treats its encoding matrix as an internal implementation detail. If the codec and the fingerprint logic use different matrices, every fingerprint check would falsely fail. Owning both ensures consistency. The codec follows the standard Vandermonde-based construction described in the Backblaze blog post and standard coding theory texts.

3.7 Multi-stage Docker build

Every Java compile happens inside a Docker eclipse-temurin:17-jdk container; the runtime stage uses the smaller eclipse-temurin:17-jre image and copies just the fat JAR. The host machine needs only Docker - no Java, no Gradle, no JAVA_HOME configuration. This makes the demo script reproducible across machines.

3.8 Two-layer verification (defense in depth)

Although the paper proposes only the homomorphic check, we layered a SHA-256 check on top of it during retrieval. SHA-256 catches bit-level corruption cheaply (256-bit hash vs polynomial evaluation over the entire shard); the homomorphic check is the authoritative consistency guarantee. The dashboard surfaces only the homomorphic result to the user, since that is the paper's contribution.

4. Implementation Details

4.1 GF(2⁸) Arithmetic

GaloisField.java precomputes EXP/LOG tables at class-load time using the primitive polynomial $x^8 + x^4 + x^3 + x^2 + 1$ (0x11d). Addition is bitwise XOR; multiplication uses $\text{EXP}[\text{LOG}[a] + \text{LOG}[b]]$ with the EXP table extended to 512 entries to avoid an explicit modulo on every multiply. Division is multiplication by the multiplicative inverse $a^{-1} = \alpha^{(255 - \log[a])}$.

4.2 Homomorphic Fingerprint

HomomorphicFingerprint.java implements the paper's Section 3 directly:

- Eval point: a non-zero random element $r \in \text{GF}(2^8)$ is chosen at upload time per object using SecureRandom. It is stored alongside the object metadata and reused on every verification.
- Polynomial evaluation: each shard is treated as a polynomial whose coefficients are its bytes. $\text{evaluate}(\text{shard}, r) = \text{shard}[0] + \text{shard}[1] \cdot r + \text{shard}[2] \cdot r^2 + \dots$ over $\text{GF}(2^8)$.
- Encoding matrix: a Vandermonde matrix is built where $V[r][c] = r^c$. The top $k \times k$ submatrix is inverted via Gauss-Jordan elimination over $\text{GF}(2^8)$, then the inverse is multiplied back to produce a systematic encoding matrix where the first k rows form an identity and the bottom $n-k$ rows give the parity coefficients.
- Expected fingerprint: for shard index j , the expected fingerprint is the linear combination $\sum M[j][i] \cdot \text{fp}(D_i)$ over $\text{GF}(2^8)$. At upload time, this is computed once per shard and stored.
- Verification: at retrieval, the fingerprint is recomputed from the actual shard bytes and compared to the stored expected value. A mismatch implies the shard is inconsistent with the original encoding.

The entire file (and its dependency GaloisField.java) uses no external libraries beyond `java.security.SecureRandom` for picking r .

4.3 Reed-Solomon Codec

ReedSolomonCodec.java performs systematic encoding using the same matrix construction. Decoding from any 3 of 5 shards proceeds by extracting the rows of the encoding matrix that correspond to the available shard indices, inverting that 3×3 matrix, and multiplying it against the available shard bytes to recover the original data.

4.4 Storage Nodes

Each storage node is a minimal Spring Boot application that exposes a gRPC service with five methods:

RPC	Purpose
StoreShard	Persist a shard to disk under /data/shards/{objectId}/{index}.shard
GetShard	Read a shard from disk
DeleteShard	Remove a shard
HealthCheck	Liveness probe
GetNodeStatus	Live counts (shards stored, bytes used, available bytes)

Shards are stored as plain files in a Docker volume, one directory per object.

4.5 Coordinator

The coordinator's ObjectStoreService handles three flows:

- Upload: encode → SHA-256 each shard → pick eval point r → compute homomorphic fingerprints → persist metadata → distribute shards via gRPC.
- Download: fetch each shard, run SHA-256 + homomorphic check, skip any that fail or time out, decode from the surviving shards (require at least 3).
- Verify: fetch every shard and run both checks, returning a per-shard PASS/FAIL/UNREACHABLE report regardless of whether reconstruction is possible.

All gRPC calls have a 500ms deadline so dead nodes never block the request thread.

4.6 Dashboard

A single index.html file served by Spring Boot renders an SVG topology of the coordinator and 5 nodes. Clicking the coordinator triggers a file upload; on response, animated dots representing each shard travel from the coordinator out to its target node. The objects table provides per-row Verify, Download, and Delete actions. Auto-refresh polls /api/nodes and /api/objects every 4 seconds. The auto-refresh is paused during uploads to prevent the SVG from being torn down mid-animation.

5. Libraries and References Used

Library	Purpose
---------	---------

Spring Boot 3.2.2	REST API, dependency injection, JPA
Spring Data JPA + Hibernate	PostgreSQL ORM
gRPC + Protobuf	Coordinator $\leftrightarrow$ node communication
PostgreSQL 15	Metadata persistence
Lombok	Boilerplate reduction (@Data, @RequiredArgsConstructor)
Springdoc OpenAPI	Swagger UI generation
Docker Compose	Multi-container orchestration

No library was used for the Reed-Solomon codec, $GF(2^8)$ arithmetic, or the homomorphic fingerprint. Those are the original contributions of the project and were implemented from the paper and standard coding theory references (Backblaze blog post on Reed-Solomon, Lin & Costello Error Control Coding).

6. Demonstration

6.1 Automated Demo Script

The script `demo-clean.sh` is a single command that exercises every system feature end-to-end. Output is concise with one or two lines per assertion.

The script demonstrates, in order:

7. Node status: all 5 nodes reachable
8. Upload: file split into 5 shards; coordinator returns objectId and eval point r
9. Verify (clean): all 5 shards PASS the homomorphic check
10. Download (clean): file is reconstructed and matches the original byte-for-byte
11. Silent corruption: one shard is overwritten via docker exec; verify shows that node FAIL while the others PASS; download still succeeds because 4 valid shards remain
12. Fault tolerance boundary: 3 nodes are stopped; download fails with "Not enough shards available to reconstruct"; one node restarted; download succeeds again from exactly 3 shards
13. Cleanup: nodes restored, demo objects deleted

6.2 Interactive Dashboard

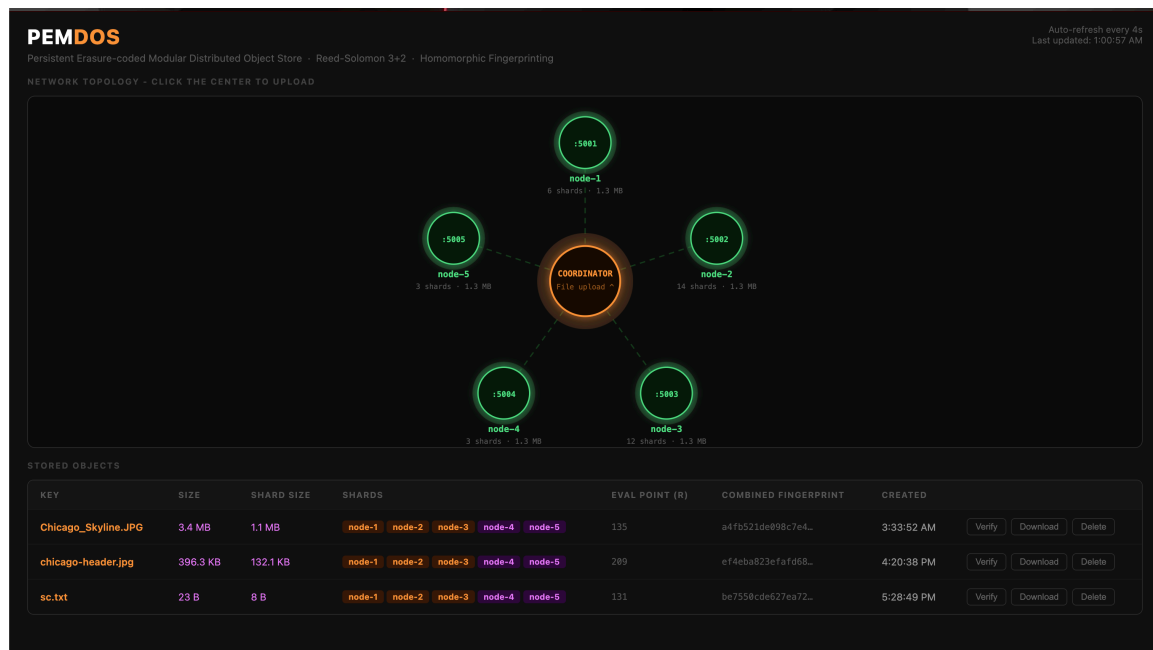
The dashboard at <http://localhost:8080> provides a visual equivalent of the script:

- Topology: a pentagon of storage nodes around the coordinator. Healthy nodes glow green; unreachable nodes glow red.
- Upload: clicking the coordinator triggers a file picker; on success, animated dots travel out to each node showing which shard went where.
- Verify panel: per-shard homomorphic results displayed as PASS/FAIL pills.
- Download / Delete: per-row buttons that surface backend errors as toast notifications.

6.3 Expected Output

On a successful run, the demo script prints:

```
— NODE STATUS
✓ node-1  HEALTHY
✓ node-2  HEALTHY
...
— VERIFY - ALL SHARDS INTACT
✓ node-1  PASS
...
— SILENT CORRUPTION DETECTION
✓ Shard 0 on node-1 corrupted - node is still alive
X node-1  FAIL  ← DETECTED
✓ node-2  PASS
...
✓ File still reconstructed correctly from 4 valid shards
— FAULT TOLERANCE - FAILURE BOUNDARY
✓ 3 nodes stopped - only 2 shards reachable
X Download failed: Not enough shards available to reconstruct
✓ node-1 restored - now exactly 3 shards reachable
✓ Download succeeded
```



7. Limitations and Future Work

- Streaming uploads: the entire file is buffered in memory before encoding (`file.getBytes()`). Files are capped at 100MB by the multipart configuration. A streaming codec would lift this limit.
- Static node membership: the 5 nodes are hard-coded in `docker-compose.yml`. Dynamic node registration (e.g. via a discovery service) would be needed for elastic scaling.

- Eval point r is stored alongside the object: in a real deployment, r would need to be kept in a separate, secured location to preserve the unpredictability the paper assumes. The current implementation stores it in the same PostgreSQL row as the rest of the metadata.
- Single coordinator: the coordinator is a single point of failure. The paper's threat model assumes the coordinator is trusted; a production system would replicate it.
- No throttling or rate limits: the API is open and unauthenticated.

8. How to Build and Run

Prerequisite: Docker Desktop (no Java, no Gradle required on the host).

Build and start:

```
docker compose up --build
```

Run the full automated demo:

```
./demo-clean.sh
```

Dashboard: <http://localhost:8080>

Swagger UI: <http://localhost:8080/swagger-ui.html>

9. Project Structure

```
PEMDOS/
├── common/                                # Shared library
│   └── src/main/
│       ├── proto/storage.proto           # gRPC service contract
│       └── java/com/pemdos/common/
│           ├── codec/ReedSolomonCodec.java # RS encode / decode
│           └── fingerprint/
│               ├── GaloisField.java       # GF(2^8) arithmetic
│               ├── HomomorphicFingerprint.java
│               └── FingerprintService.java # SHA-256 helpers
├── storage-node/                          # gRPC storage server
├── coordinator/                           # REST API+dashboard+metadata
│   └── src/main/
│       ├── java/com/pemdos/coordinator/...
│       └── resources/
│           ├── application.yml
│           └── static/index.html          # Dashboard UI
├── docker-compose.yml                     # psql+5 nodes+ coordinator
├── demo.sh                               # Verbose demo script
└── demo-clean.sh                         # Concise demo script
```